Cairo 플러그인 시스템과 확장성

개요

Cairo 컴파일러는 강력한 플러그인 시스템을 통해 확장성을 제공합니다. 이 시스템은 매크로 플러그인(Macro Plugin)과 인라인 매크로 플러그인(Inline Macro Plugin)을 통해 컴파일 시점에 코드를 생성하고 변환할 수 있는 기능을 제공합니다. 플러그인 시스템은 `cairo-lang-plugins` crate에서 구현되며, 다양한 내장 플러그인들이 포함되어 있습니다.

플러그인 아키텍처

핵심 트레이트

Cairo 플러그인 시스템의 핵심은 두 가지 주요 트레이트로 구성됩니다:

MacroPlugin 트레이트

```
pub trait MacroPlugin: std::fmt::Debug + Sync + Send + Any {
/// 아이템에 대한 코드를 생성합니다
fn generate_code<'db>(
&self,
db: &'db dyn Database,
item_ast: ast::ModuleItem<'db>,
metadata: &MacroPluginMetadata<'_>,
) -> PluginResult<'db>;
```

```rust
/// 플러그인이 사용하는 속성들을 선언합니다
fn declared_attributes(&self) -> Vec<String>;

/// 플러그인이 제공하는 derive들을 선언합니다
fn declared_derives(&self) -> Vec<String> { Vec::new() }

/// 실행 가능한 함수를 표시하는 속성들
fn executable_attributes(&self) -> Vec<String> { Vec::new() }

/// 팬텀 타입을 표시하는 속성들
fn phantom_type_attributes(&self) -> Vec<String> { Vec::new() }

/// 플러그인의 타입 ID
fn plugin_type_id(&self) -> any::TypeId { self.type_id() }
}
```

InlineMacroExprPlugin 트레이트

```rust
pub trait InlineMacroExprPlugin: std::fmt::Debug + Sync + Send + Any {
/// 인라인 매크로에 대한 코드를 생성합니다
fn generate_code<'db>(
&self,
db: &'db dyn Database,
```

```
item_ast: &ast::ExprInlineMacro<'db>,
metadata: &MacroPluginMetadata<'_>,
) -> InlinePluginResult<'db>;


/// 인라인 매크로에 대한 문서를 제공합니다
fn documentation(&self) -> Option<String> { None }


/// 플러그인의 타입 ID
fn plugin_type_id(&self) -> any::TypeId { self.type_id() }
}
```

플러그인 메타데이터

플러그인은 실행 시 다음과 같은 메타데이터를 받습니다:

```
pub struct MacroPluginMetadata<'a> {
/// 현재 crate의 설정 집합
pub cfg_set: &'a CfgSet,
/// 다른 플러그인들이 선언한 derive들
pub declared_derives: &'a OrderedHashSet<String>,
/// 매크로 활성화 지점에서 허용된 기능들
pub allowed_features: &'a OrderedHashSet<&'a str>,
/// 현재 crate의 에디션
pub edition: Edition,
```

```
}
```

플러그인 결과

플러그인의 실행 결과는 다음 구조체로 반환됩니다:

```
pub struct PluginResult<'db> {
/// 생성된 코드 파일
pub code: Option<PluginGeneratedFile>,
/// 진단 메시지들
pub diagnostics: Vec<PluginDiagnostic<'db>>,
/// 원본 아이템을 제거할지 여부
pub remove_original_item: bool,
}

pub struct PluginGeneratedFile {
/// 가상 파일의 이름
pub name: String,
/// 파일의 코드 내용
pub content: String,
/// 코드 매핑 정보
pub code_mappings: Vec<CodeMapping>,
/// 플러그인이 생성한 추가 데이터
pub aux_data: Option<DynGeneratedFileAuxData>,
```

```
/// 진단 노트
pub diagnostics_note: Option<String>,
/// 비위생적(unhygienic) 플러그인 여부
pub is_unhygienic: bool,
}
```

플러그인 등록 메커니즘

기본 플러그인 로딩

Cairo 컴파일러는 `get_base_plugins()` 함수를 통해 기본 플러그인들을 로드합니다:

```
pub fn get_base_plugins() -> Vec<Arc<dyn MacroPlugin>> {
vec![
Arc::new(ConfigPlugin::default()),
Arc::new(DerivePlugin::default()),
Arc::new(GenerateTraitPlugin::default()),
Arc::new(PanicablePlugin::default()),
Arc::new(CompileErrorPlugin::default()),
Arc::new(ExternalAttributesValidationPlugin::default()),
]
}
```

플러그인 인터닝

플러그인들은 Salsa 데이터베이스에 인터닝되어 효율적으로 관리됩니다:

```
#[derive(Clone, Debug)]
pub struct MacroPluginLongId(pub Arc<dyn MacroPlugin>);


define_short_id!(MacroPluginId, MacroPluginLongId, DefsGroup);
```

플러그인 설정

플러그인들은 crate별로 설정할 수 있으며, 기본 플러그인이나 오버라이드된 플러그인을 사용할 수 있습니다:

```
pub trait DefsGroup: Database {
fn default_macro_plugins<'db>(&'db self) -> &'db [MacroPluginId<'db>];
fn macro_plugin_overrides<'db>(&'db self) -> &'db OrderedHashMap<CrateId<'db>,
fn crate_macro_plugins<'db>(&'db self, crate_id: CrateId<'db>) -> &'db [MacroPl
}
```

내장 플러그인들

1. ConfigPlugin

조건부 컴파일을 위한 `#[cfg(...)]` 속성을 처리합니다.

주요 기능:

- `#[cfg(test)]`, `#[cfg(feature = "...")]` 등의 조건부 컴파일

- 복잡한 논리 연산자 지원: `not`, `and`, `or`
- 중첩된 조건 평가

구현 예시:

```
impl MacroPlugin for ConfigPlugin {
fn  generate_code<'db>(&self, db: &'db dyn Database, item_ast: ast::ModuleItem<'
if  should_drop(db, metadata.cfg_set, &item_ast, &mut  diagnostics) {
PluginResult { remove_original_item: true, .. }
} else {
// 아이템 유지 또는 부분 수정
}
}
}
```

2. DerivePlugin

자동 트레이트 구현을 위한 `#[derive(...)]` 속성을 처리합니다.

지원하는 derive들:

- `Copy`, `Drop`: 빈 구현 생성
- `Clone`: 깊은 복사 구현
- `Debug`: 디버그 출력 구현
- `Default`: 기본값 생성 구현
- `Destruct`: 소멸자 구현
- `Hash`: 해시 함수 구현

- `PanicDestruct` : 패닉 소멸자 구현
- `PartialEq` : 부분 동등성 비교 구현
- `Serde` : 직렬화/역직렬화 구현

구현 패턴:

```
fn generate_derive_code_for_type<'db>(
db: &'db dyn Database,
metadata: &MacroPluginMetadata<'_>,
info: PluginTypeInfo<'db>,
) -> PluginResult<'db> {
let mut builder = PatchBuilder::new(db, &info.attributes);
for derived in parsed_derives {
let code = match derived {
"Clone" => clone::handle_clone(&info),
"Debug" => debug::handle_debug(&info),
// ... 다른 derive들
};
builder.add_modified(RewriteNode::mapped_text(code, db, &derived_path));
}
let (content, code_mappings) = builder.build();
// PluginResult 반환
}
```

3. GenerateTraitPlugin

`#[generate_trait]` 속성을 통해 impl에서 trait을 자동 생성합니다.

사용 예시:

```
#[generate_trait]

impl MyTraitImpl of MyTrait {

fn my_function(self: @MyType) -> u32 { ... }

}
```

생성되는 코드:

```
trait MyTrait {

fn my_function(self: @MyType) -> u32;

}
```

4. PanicablePlugin

함수가 패닉할 수 있음을 표시하는 속성들을 처리합니다.

5. CompileErrorPlugin

컴파일 시점에 사용자 정의 에러를 발생시키는 `#[compile_error("message")]` 속성을 처리합니다.

6. ExternalAttributesValidationPlugin

외부 플러그인에서 선언되지 않은 속성들에 대한 검증을 수행합니다.

매크로 시스템

절차적 매크로 (Procedural Macros)

Cairo의 플러그인 시스템은 절차적 매크로와 유사한 기능을 제공합니다. 플러그인들은 AST를 입력받아 새로운 코드를 생성할 수 있습니다.

특징:

- 컴파일 시점에 실행
- AST 기반 코드 변환
- 타입 안전성 보장
- 진단 메시지 생성 가능

선언적 매크로 (Declarative Macros)

현재 Cairo는 주로 속성 기반의 선언적 매크로를 지원합니다:

```
#[derive(Clone, Debug)]

struct MyStruct {

field: u32,

}



#[cfg(test)]

mod tests {

// 테스트 코드

}
```

커스텀 속성과 코드 생성

속성 처리

플러그인들은 다양한 형태의 속성을 처리할 수 있습니다:

```
// 단순 속성
#[my_attribute]

// 인수가 있는 속성
#[my_attribute(arg1, arg2)]

// 키-값 속성
#[my_attribute(key = "value")]

// 복잡한 속성
#[my_attribute(nested(inner = "value"))]
```

코드 생성 패턴

플러그인들은 `PatchBuilder` 를 사용하여 코드를 생성합니다:

```
let mut builder = PatchBuilder::new(db, &original_item);

// 기존 노드 추가
builder.add_node(item.as_syntax_node());
```

```
// 수정된 텍스트 추가
builder.add_modified(RewriteNode::mapped_text(generated_code, db, &reference_nod

// 문자열 직접 추가
builder.add_str("impl MyTrait for MyType {\n");

let (content, code_mappings) = builder.build();
```

타입 정보 활용

플러그인들은 `PluginTypeInfo` 를 통해 타입 정보를 활용합니다:

```
pub struct PluginTypeInfo<'a> {
pub name: &'a str, // 타입 이름
pub attributes: ast::AttributeList<'a>, // 속성 목록
pub generics: GenericParamsInfo<'a>, // 제네릭 정보
pub members_info: Vec<MemberInfo<'a>>, // 멤버 정보
pub type_variant: TypeVariant, // 구조체/열거형 구분
}
```

플러그인 개발 가이드라인

1. 기본 플러그인 구조

```rust
#[derive(Debug, Default)]
pub struct MyPlugin;

impl MacroPlugin for MyPlugin {
fn generate_code<'db>(
&self,
db: &'db dyn Database,
item_ast: ast::ModuleItem<'db>,
metadata: &MacroPluginMetadata<'_>,
) -> PluginResult<'db> {
// 플러그인 로직 구현
PluginResult::default()
}

fn declared_attributes(&self) -> Vec<String> {
vec!["my_attribute".to_string()]
}
}
```

2. 속성 파싱

```rust
use cairo_lang_syntax::attribute::structured::AttributeStructurize;
```

```
let attr = item_ast.find_attr(db, "my_attribute")?;
let structured_attr = attr.structurize(db);

for arg in structured_attr.args {
match arg.variant {
AttributeArgVariant::Unnamed(expr) => {
// 이름 없는 인수 처리
}
AttributeArgVariant::Named { name, value } => {
// 이름 있는 인수 처리
}
}
}
```

3. 진단 메시지 생성

```
let diagnostics = vec![
PluginDiagnostic::error(
stable_ptr,
"Invalid attribute usage".to_string()
),
PluginDiagnostic::warning(
stable_ptr,
```

```
"This usage is deprecated".to_string()
),
];
```

4. 코드 생성 모범 사례

위생적 매크로 작성:

- 생성된 코드에서 변수명 충돌 방지
- 적절한 스코프 사용
- 네임스페이스 충돌 방지

에러 처리:

- 명확하고 도움이 되는 에러 메시지
- 적절한 stable pointer 사용
- 복구 가능한 에러 처리

성능 고려사항:

- 불필요한 코드 생성 방지
- 효율적인 AST 순회
- 캐싱 활용

5. 테스트 작성

```
#[test]
fn test_my_plugin() {
let mut db = TestDatabase::default();
```

```
let plugin = MyPlugin::default();

let cairo_code = indoc! {r#"

#[my_attribute]

struct TestStruct {

field: u32,

}

"#};

// 테스트 로직

}
```

확장성과 호환성

플러그인 체인

플러그인들은 순서대로 실행되며, 각 플러그인의 결과가 다음 플러그인의 입력이 될 수 있습니다:

1. `ConfigPlugin` : 조건부 컴파일로 아이템 필터링
2. `DerivePlugin` : 트레이트 구현 생성
3. `GenerateTraitPlugin` : 트레이트 정의 생성
4. 기타 플러그인들

버전 호환성

플러그인 시스템은 Cairo 컴파일러의 버전과 밀접하게 연관되어 있습니다:

- AST 구조 변경에 대한 대응
- 새로운 언어 기능 지원

- 하위 호환성 유지

외부 플러그인 지원

현재 Cairo는 내장 플러그인들을 주로 지원하지만, 향후 외부 플러그인 지원을 위한 확장이 가능합니다:

- 동적 플러그인 로딩
- 플러그인 레지스트리
- 보안 샌드박싱

결론

Cairo의 플러그인 시스템은 언어의 확장성과 유연성을 크게 향상시킵니다. 강력한 매크로 시스템을 통해 반복적인 코드 작성을 줄이고, 도메인 특화 언어 기능을 추가할 수 있습니다. 잘 설계된 아키텍처와 명확한 인터페이스를 통해 안전하고 효율적인 코드 생성이 가능하며, 이는 Cairo 생태계의 발전에 중요한 역할을 합니다.